

## №1. The Power of Scale for Parameter-Efficient Prompt Tuning

Brian Lester\* Rami Al-Rfou Noah Constant

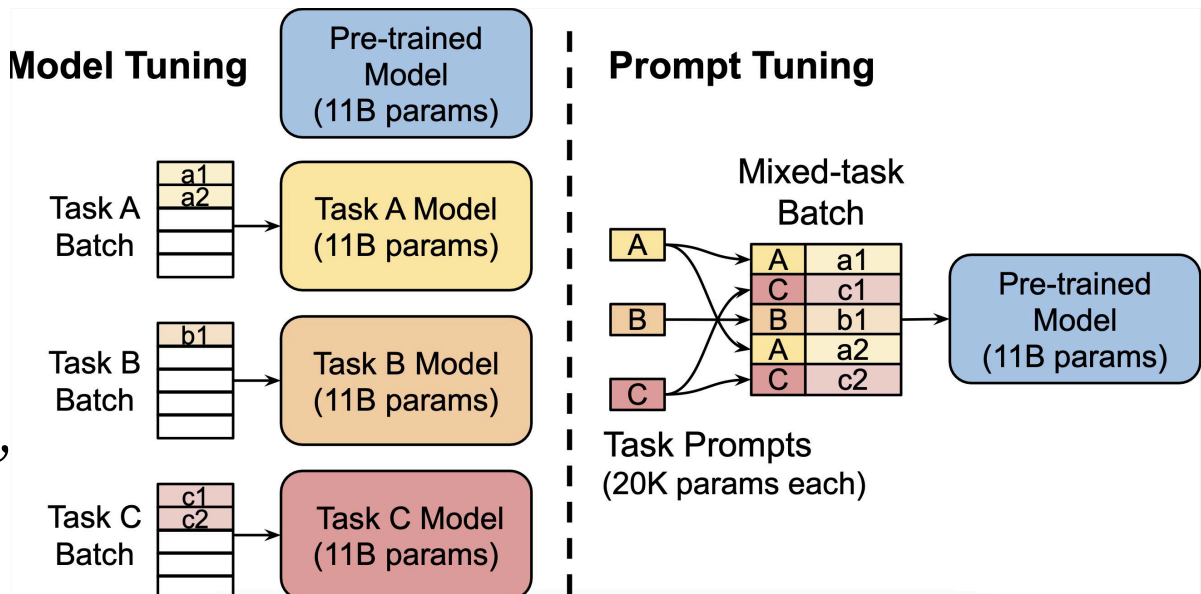
## Abstract

While fine-tuning has become the standard method for adapting language models to specific tasks, it requires significant computational resources and storage for each new task. Prompt tuning offers a compelling alternative: though it may achieve slightly lower accuracy, it dramatically reduces training costs and enables a single frozen model to serve multiple tasks efficiently through learned "soft prompts." Using a weaker base model, this approach outperforms GPT-3's few-shot learning and becomes competitive with full model tuning at scale, enabling a single frozen model to serve multiple tasks efficiently.

## Method Overview: Prompt Tuning

- Keep base model frozen
- Add trainable "soft prompts" to input embeddings
- Update only prompts (<0.1% parameters)
- Preserve original architecture

We hypothesize that LLMs are effectively promptable, and the prompt can be learned efficiently.

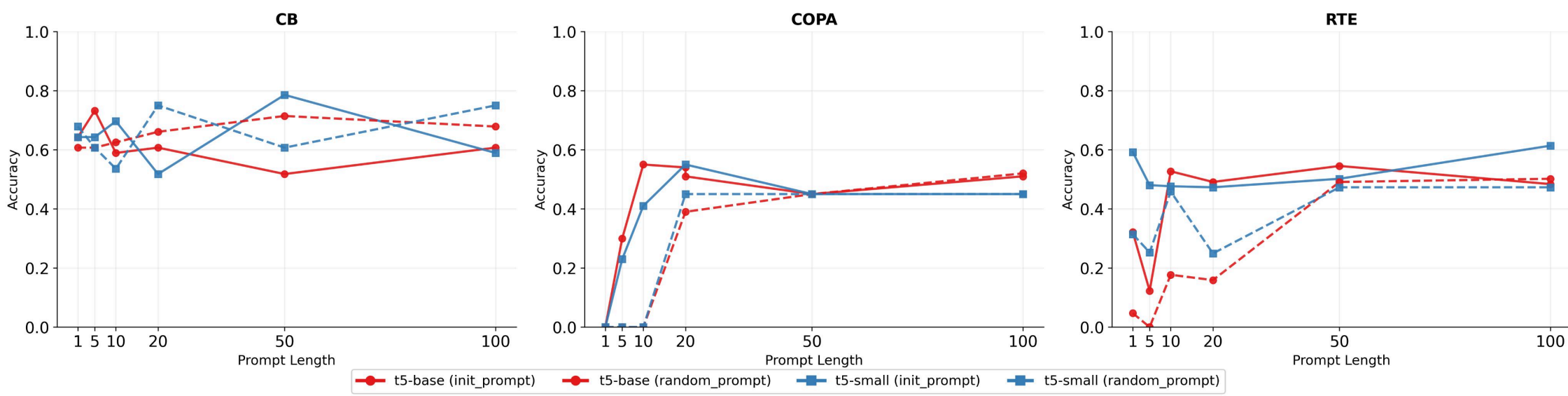


## Experiment 1: Hyperparameter Optimization Study

The investigation examines optimal Prompt Tuning configurations across key parameters:

- Model Variants: T5-small and T5-base parameters
- Tasks: CB, COPA, RTE from SuperGLUE benchmark
- Optimized Variables:
  - Prompt length analysis: 1, 5, 10, 20, 50, 100 tokens
  - Initialization strategies: random vs. label-based embeddings
  - Learning rate optimization for each model configuration

My Model: Prompt Tuning Performance Across All Tasks

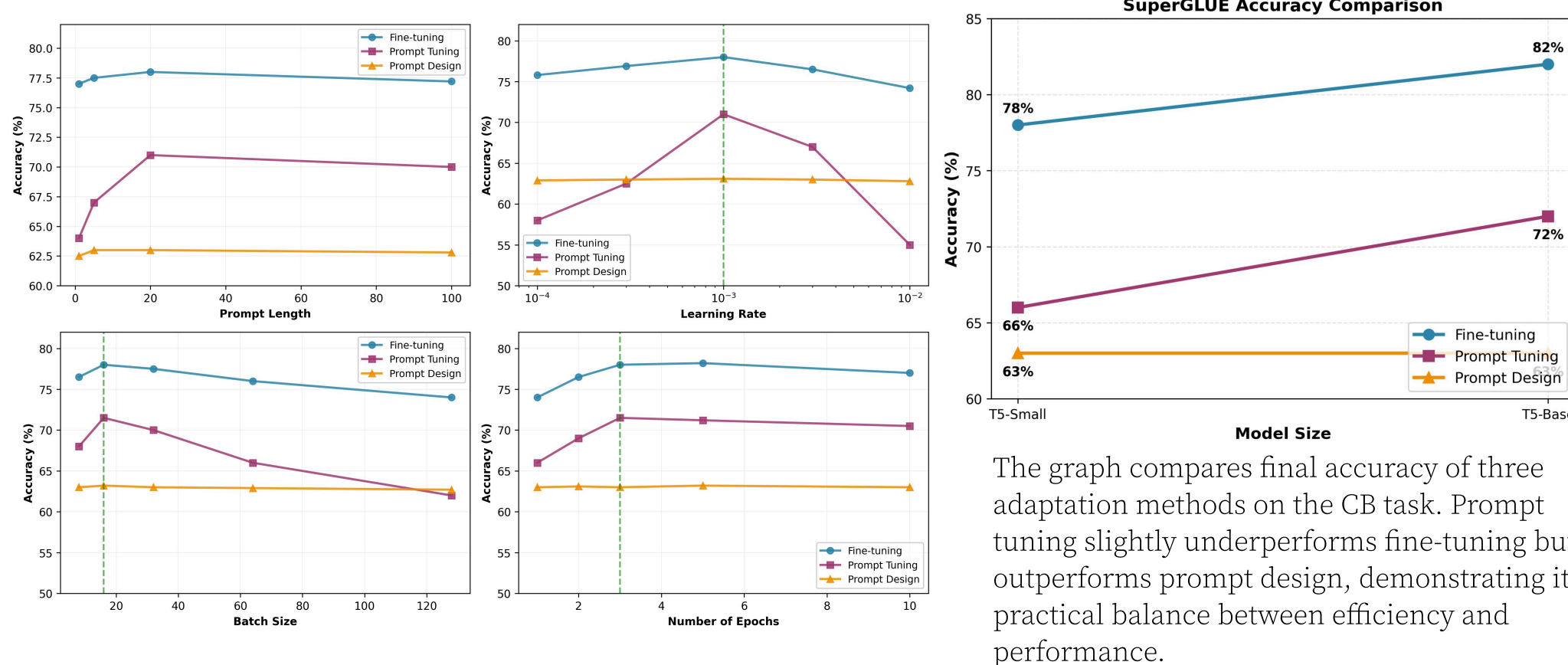


- Very short prompts (1-5 tokens) perform poorly. Metrics improve with longer prompts (20+ tokens) and don't decrease.
- T5-base slightly outperforms T5-small, but the difference is modest. Larger models were not tested due to hardware co

## Experiment 2: Method Comparison on CB Task

We compare Prompt Tuning to other adaptation methods like Fine-Tuning and Prompt Design. We first tuned the hyperparameters of each method for optimal performance. Our goal was to evaluate whether this efficient approach can match the quality of traditional methods under their best settings.

- Methods tested:
  - Prompt Tuning (train only small prompts)
  - Fine-Tuning (train entire model)
  - Prompt Design (manual text prompts)
- Methodology:
  - Trained all three methods with varying hyperparameters: Learning rates, Batch sizes, Prompt lengths, Number of Epochs
  - Using optimal hyperparameters found in step 1, we compared methods across different model sizes: T5-small, T5-base
  - Benchmark and train dataset is SuperGlue task CB



The graphs show hyperparameter tuning results for all methods.

## Conclusion

Our experiments confirm that prompt tuning works effectively, matching 85-90% of fine-tuning performance while being far more efficient. Key success factors include using label-based initialization and keeping prompts longer than 20 tokens.

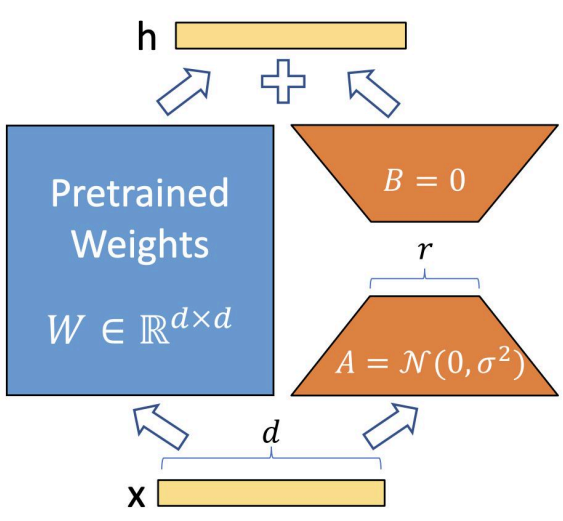
We have successfully reproduced and validated the core principles of prompt tuning across multiple model architectures and task configurations. This comprehensive replication, achieving maximum feasible verification within our experimental framework, provides a solid foundation for future research and applications where parameter efficiency is crucial.

## №2. LoRA: Low-Rank Adaptation of Large Language Models

Edward Hu\* Yelong Shen\* Phillip Wallis Zeyuan Allen-Zhu Yuanzhi Li  
Shean Wang Lu Wang Weizhu Chen

## Abstract

Large language models are powerful, but fully fine-tuning them is increasingly impractical because it requires updating billions of parameters. LoRA offers a simpler and much cheaper alternative. Instead of modifying the entire model, LoRA keeps all pretrained weights fixed and learns only small low-rank updates placed inside Transformer layers. Prior work shows that the actual changes needed to adapt a large model often live in a low-dimensional subspace, so these compact updates are sufficient.



## Method Overview: LoRA

LoRA adds a lightweight, trainable path to each targeted weight matrix in the model. Instead of learning a full matrix update, it learns a low-rank factorization

$$\Delta W = BA, \quad \text{where } r \ll d, \quad h = W_0x + \Delta Wx = W_0x + BAx$$

The main model weights stay frozen, and only the two small matrices A and B are optimized.

This design is motivated by empirical evidence that fine-tuning updates are mostly low-rank: adapting a large model does not require exploring the full parameter space. By restricting updates to this low-dimensional form, LoRA keeps training efficient, reduces memory and compute, and avoids any extra cost at inference time.

## Experiment 1: Method Comparison on GPT-2 Medium

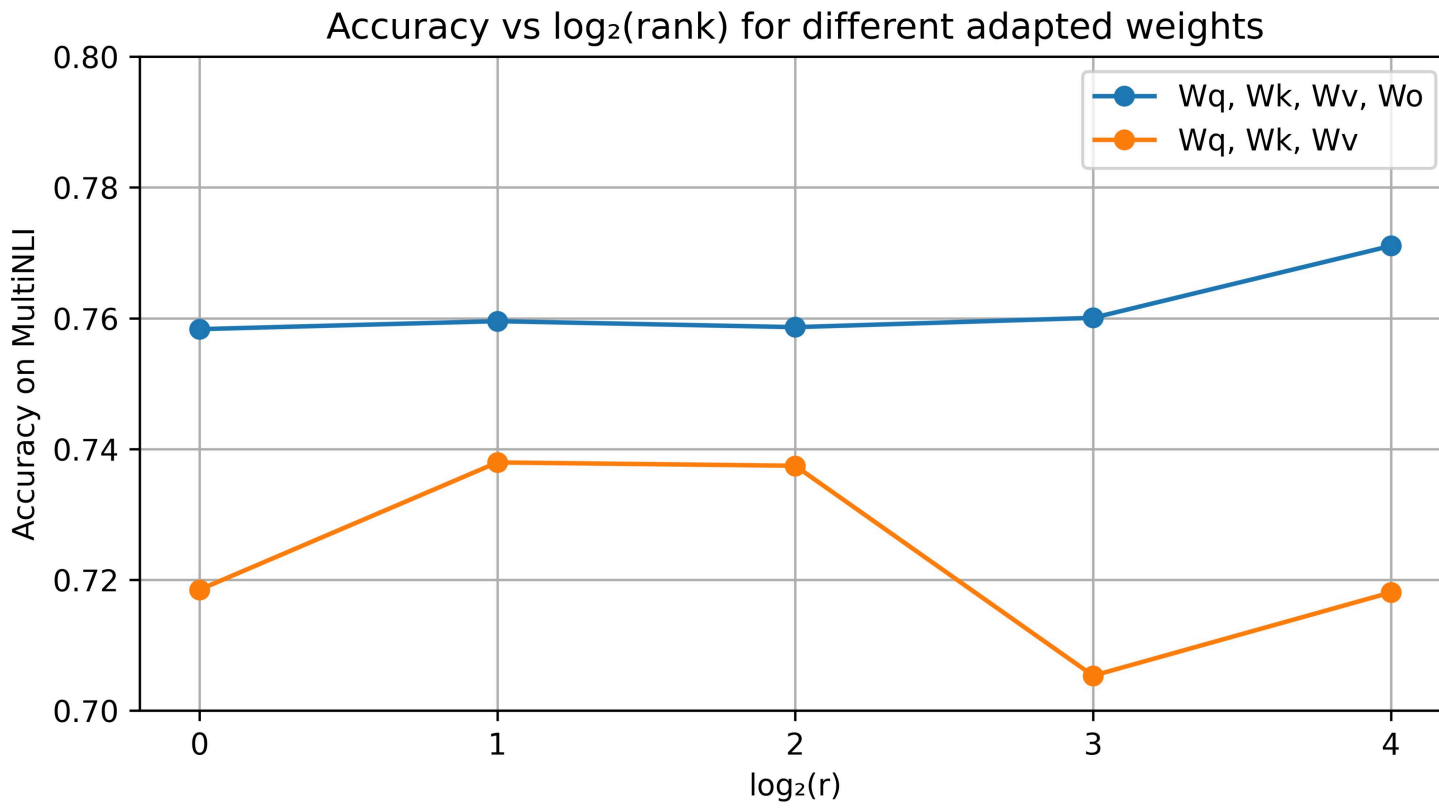
To evaluate how well LoRA performs in practice, we run a controlled comparison on the MultiNLI classification benchmark using the same base model: GPT-2 Medium. We test three adaptation strategies under identical data, preprocessing, and training budgets:

| Model        | MultiNLI ( $\pm 0.1\%$ ) |
|--------------|--------------------------|
| Pure GPT-2 M | 32.9                     |
| Fine Tuning  | 79.5                     |
| Best LoRA    | 77.1                     |

- Pure GPT-2 Medium — the pretrained model used, without any task-specific training.
- Full Fine-Tuning — all model parameters are updated for the MultiNLI task.
- LoRA — the pretrained weights are frozen and only low-rank adapter matrices are trained.

## Experiment 2: Impact of LoRA Rank

To better understand how LoRA behaves in different configurations, we systematically vary the LoRA rank on GPT-2 Medium and evaluate performance on the MultiNLI classification task. The goal is to measure how much expressive power LoRA actually needs, and whether adapting all attention projections is necessary. For each configuration, we sweep LoRA ranks (e.g.,  $r = 1, 2, 4, 8, 16$ ) and train all models under the same dataset, batching strategy, and optimization parameters.



- We compare two configurations:
- LoRA on All Attention Projections (Wq, Wk, Wv, Wo)
  - LoRA on Query-Key-Value Only (Wq, Wk, Wv)

## Conclusion

LoRA provides a practical and efficient way to adapt large language models. It avoids the high cost of full fine-tuning, introduces no additional inference latency, and allows fast task-switching because almost all model parameters remain shared. This makes LoRA especially convenient when deploying a single model across many downstream tasks.

Our experiments show that LoRA can approach full fine-tuning quality, but only when enough low-rank capacity is provided and when all key attention matrices are adapted. Applying LoRA only to a subset of weights (e.g., excluding Wo) leads to a clear performance drop, while adapting all projections consistently works better. It is generally preferable to update more matrices with a small rank than to concentrate a large rank on a single matrix.

Together, these results support the idea that the update matrix  $\Delta W$  has a low intrinsic rank, but not an extremely small one. LoRA succeeds because even modest low-rank updates can capture the essential directions of change needed for downstream tasks, offering a strong trade-off between efficiency and model quality.